Loading

This section covers **the Pintos loader** and **basic kernel initialization**.

The Loader

The first part of Pintos that runs is the loader, in `threads/loader.S`.

- **The PC BIOS** loads the loader into memory.
- The loader, in turn, is responsible for finding the kernel on disk, loading it into memory, and then jumping to its start.

It's not important to understand exactly how the loader works, but if you're interested, read on. You should probably read along with the loader's source.

The PC BIOS loads the loader from the first sector of the first hard disk, called the *master boot record* (MBR).

- PC conventions reserve 64 bytes of the MBR for the partition table, and Pintos uses about 128 additional bytes for kernel command-line arguments.
- This leaves **a little over 300 bytes for the loader's own code**. This is a severe restriction that means, practically speaking, the loader must be written in assembly language.

The Pintos loader and kernel don't have to be on the same disk, nor is the kernel required to be in any particular location on a given disk.

- **The loader's first job, then, is to find the kernel** by reading the partition table on each hard disk, looking for a bootable partition of the type used for a Pintos kernel.

Low-Level Kernel Initialization

The loader's last action is to transfer control to the kernel's entry point, which is `start()` in `threads/start.S`. The job of this code is to switch the CPU from legacy 16-bit **"real mode"** into the 32-bit **"protected mode"** used by all modern 80×86 operating systems.

1. The startup code's first task is actually to **obtain the machine's memory size**, by asking the BIOS for the PC's memory size. The simplest BIOS function to do this can only detect **up to 64 MB of RAM**, so that's the practical limit that Pintos can support. The function stores the memory size, in pages, in global variable `init_ram_pages`.
2. The first part of CPU initialization is to **enable the A20 line**, that is, the CPU's address line numbered 20. For historical reasons, PCs boot with this address line fixed at 0, which means that attempts to access memory beyond the first 1 MB (2 raised to the 20th power) will fail. Pintos wants to access more memory than this, so we have to enable it.
3. Next, the loader **creates a basic page table**.
    - This page table maps the 64 MB at the base of virtual memory (starting at virtual address 0) directly to the identical physical addresses.
    - It also maps the same physical memory starting at virtual address `LOADER_PHYS_BASE`, which defaults to `0xc0000000` (3 GB).
    - The Pintos kernel only wants the latter mapping, but there's a chicken-and-egg problem if we don't include the former: our current virtual address is roughly 0x20000, the location where the loader put us, and we can't jump to 0xc0020000 until we turn on the page table, but if we turn on the page table without jumping there, then we've just pulled the rug out from under ourselves.

4. After the page table is initialized, we **load the CPU's control registers to turn on protected mode and paging, and set up the segment registers**. We aren't yet equipped to handle interrupts in protected mode, so we **disable interrupts**.
5. The final step is to **call `pintos_init()`**.

High-Level Kernel Initialization

The kernel proper starts with the `pintos_init()` function. The `pintos_init()` function is written in C, as will be most of the code we encounter in Pintos from here on out.

1. **When `pintos_init()` starts, the system is in a pretty raw state.** We're in 32-bit protected mode with paging enabled, but hardly anything else is ready. Thus, the `pintos_init()` function consists primarily of calls into other Pintos modules' initialization functions. These are usually named `module_init()`, where module is the module's name, `module.c` is the module's source code, and `module.h` is the module's header.
2. The first step in `pintos_init()` is to call **`bss_init()`**, which **clears out the kernel's "BSS"**, which is the traditional name for a segment that should be initialized to all zeros. In most C implementations, whenever you declare a variable outside a function without providing an initializer, that variable goes into the BSS. Because it's all zeros, the BSS isn't stored in the image that the loader brought into memory. We just use `memset()` to zero it out.
3. Next, `pintos_init` calls **`read_command_line()`** to **break the kernel command line into arguments**, then **`parse_options()`** to **read any options at the beginning of the command line**. (Actions specified on the command line execute later.)
4. **`thread_init()` initializes the thread system.** We will defer full discussion to our discussion of Pintos threads in the Threads section. It is called so early in initialization because a valid thread structure is a prerequisite for acquiring a lock, and lock acquisition in turn is important to other Pintos subsystems.

5. Then we **initialize the console** and **print a startup message to the console**.
6. The next block of functions we call **initializes the kernel's memory system**.
    - `palloc_init()` sets up the kernel page allocator, which doles out memory one or more pages at a time (see section Page Allocator).
    - `malloc_init()` sets up the allocator that handles allocations of arbitrary-size blocks of memory (see section Block Allocator).
    - `paging_init()` sets up a page table for the kernel (see section Page Table).
7. In projects 2 and later, `pintos_init()` also calls `tss_init()` and `gdt_init()`.
8. The next set of calls **initializes the interrupt system**.
    - `intr_init()` sets up the CPU's *interrupt descriptor table* (IDT) to ready it for interrupt handling (see section Interrupt Handling),
    - then `timer_init()` and `kbd_init()` prepare for handling timer interrupts and keyboard interrupts, respectively.
    - `input_init()` sets up to merge serial and keyboard input into one stream.
    - In projects 2 and later, we also prepare to handle interrupts caused by user programs using `exception_init()` and `syscall_init()`.
9. Now that interrupts are set up, we can **start the scheduler with** `thread_start()`, which **creates the idle thread** and **enables interrupts**.
10. With interrupts enabled, interrupt-driven serial port I/O becomes possible, so we use `serial_init_queue()` to switch to that mode.
11. Finally, `timer_calibrate()` **calibrates the timer for accurate short delays**.
12. If the file system is compiled in, as it will starting in project 2, we **initialize the IDE disks** with `ide_init()`, then **the file system** with `filesys_init()`.
13. **Boot is complete, so we print a message.**
14. Function `run_actions()` now **parses and executes actions specified on the kernel command line**, such as `run` to run a test (in project 1) or a user program (in later projects).

15. Finally, if **-q** was specified on the kernel command line, we call **`shutdown_power_off()`** to terminate the machine simulator. **Otherwise**, `pintos_init()` calls **`thread_exit()`**, which allows any other running threads to continue running.

Physical Memory Map

Memory Range (0x)	Owner	Contents
00000000--000003ff	CPU	Real mode interrupt table.
00000400--000005ff	BIOS	Miscellaneous data area.
00000600--00007bff	--	---
00007c00--00007dff	Pintos	Loader.
0000e000--0000efff	Pintos	Stack for loader; kernel stack and `struct thread` for initial kernel thread.
0000f000--0000ffff	Pintos	Page directory for startup code.
00010000--00020000	Pintos	Page tables for startup code.
00020000--0009ffff	Pintos	Kernel code, data, and uninitialized data segments.
000a0000--000bffff	Video	VGA display memory.
000c0000--000effff	Hardware	Reserved for expansion card RAM and ROM.
000f0000--000fffff	BIOS	ROM BIOS.
00100000--03ffffff	Pintos	Dynamic memory allocation.

i.e.:


+------------------+ <- 0xFFFFFFFF (4GB)
| 32-bit |
| memory mapped |
| devices |
| |
/\/\/\/\/\/\/\/\/\
/\/\/\/\/\/\/\/\/\
| |
| Unused |
| |
+------------------+ <- depends on amount of RAM
| |
| |
| Extended Memory |
| |
| |
+------------------+ <- 0x00100000 (1MB)
| BIOS ROM |
+------------------+ <- 0x000F0000 (960KB)
| 16-bit devices, |
| expansion ROMs |
+------------------+ <- 0x000C0000 (768KB)
| VGA Display |
+------------------+ <- 0x000A0000 (640KB)
| pintos kernel |
+------------------+ <- 0x00020000 (128KB)
| page tables |
| for startup |
+------------------+ <- 0x00010000 (64KB)
| page directory |
| for startup |
+------------------+ <- 0x0000f000 (60KB)
| initial kernel |
| thread struct |
+------------------+ <- 0x0000e000 (56KB)
| / |
+------------------+ <- 0x00007e00 (31.5KB)
| pintos loader |
+------------------+ <- 0x00007c00 (31KB)
| / |
+------------------+ <- 0x00000600 (1536B)
| BIOS data |
+------------------+ <- 0x00000400 (1024B)
| CPU-owned |
+------------------+ <- 0x00000000

Previous
Code Guide

Next
Threads

Last updated 1 year ago